

Software Engineering - M5 Database Management (M5-A1)

Section A – Concept Application

S1.

Justify your constraint choices for at least three columns across two tables (restaurants, menu items, orders). What data integrity problems would arise if these constraints were not applied?

Answer:

1. restaurants.restaurant_id as PRIMARY KEY: it uniquely identifies each restaurant, so every other table (menu_items, orders) can reference it reliably. Without a primary key, duplicate or ambiguous restaurant records could exist, and joins based on restaurant_id could match the wrong row or multiple rows unexpectedly. 2. restaurants.name as NOT NULL: a restaurant record with no name is useless to display anywhere in the app. Without this constraint, blank or NULL restaurant names could slip into the database and break the UI or search functionality silently. 3. menu_items.restaurant_id as FOREIGN KEY referencing restaurants.restaurant_id: this guarantees every menu item belongs to a restaurant that actually exists. Without it, a menu item could reference a restaurant_id that was deleted or never existed (an 'orphan' row), causing broken joins, missing restaurant names in the menu display, and inconsistent reports. In general, skipping these constraints trades short-term convenience for long-term data corruption - duplicate keys, orphaned foreign references, and missing required fields compound over time and are far harder to clean up after the fact than to prevent upfront.

S2.

Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?

Answer:

The WHERE clause is what limits an UPDATE or DELETE to the specific rows that should be affected. Without it, MySQL treats the statement as applying to every row in the table. For the 'Pending' orders older than 7 days: the WHERE clause (status = 'Pending' AND order_date < NOW() - INTERVAL 7 DAY) ensures only genuinely stale pending orders are cancelled. Without it, UPDATE orders SET status = 'Cancelled' would cancel every order in the entire table - including ones placed a minute ago or already delivered. For deleting unavailable menu items from closed restaurants: the WHERE clause (status = 'Unavailable' AND restaurant is closed) ensures only that specific subset is removed. Without it, DELETE FROM menu_items would wipe out the entire menu_items table, including items from restaurants that are still open and operating. In short, a missing WHERE condition turns a targeted fix into a full-table operation - and since UPDATE/DELETE changes are usually only reversible via a transaction ROLLBACK (before COMMIT) or a database backup, this kind of mistake can cause irreversible, business-critical data loss in production.

S3.

Which SQL clauses and aggregate functions would you use to produce a revenue report (total revenue, average order value, orders per restaurant) but only for restaurants with more than 20 orders this month? Explain why HAVING must be used rather than WHERE to filter by aggregate results.

Answer:

This report needs GROUP BY restaurant_id to bucket orders per restaurant, along with the aggregate functions SUM(total_amount) for total revenue, AVG(total_amount) for average order value, and COUNT(*) for number of orders. A WHERE clause on order_date filters which raw rows enter the grouping (e.g., only this month's orders), while HAVING filters the grouped results afterward. HAVING is required (not WHERE) for the 'more than 20 orders' condition because COUNT(*) > 20 is a value that only exists after grouping and aggregation have happened - WHERE is evaluated row-by-row before GROUP BY runs, so it has no access to aggregate results like COUNT(*), SUM(), or AVG(). HAVING runs after the GROUP BY stage specifically so it can filter based on those computed aggregate values.

```
SELECT
    restaurant_id,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS avg_order_value
FROM orders
WHERE order_date >= DATE_FORMAT(NOW(), '%Y-%m-01') -- current month's orders only
GROUP BY restaurant_id
HAVING COUNT(*) > 20;
```

S4.

Compare INNER JOIN and LEFT JOIN for the order history page. Which would you use and why? Describe what each join type does to records that have no match in the other table.

Answer:

INNER JOIN only returns rows where a match exists in both tables - if an order's restaurant_id doesn't exist in the restaurants table anymore, that order would be silently dropped from the results entirely. LEFT JOIN returns every row from the left table (orders) regardless of whether a match exists in the right table (restaurants); when there's no match, the columns coming from restaurants (like name) are filled with NULL instead of the row being excluded. For this order history page, LEFT JOIN is the correct choice, since the requirement explicitly says to show every order 'including any that reference a restaurant no longer in the table.' Using INNER JOIN here would quietly hide orders tied to deleted restaurants, giving customers an incomplete and inaccurate order history.

```
-- LEFT JOIN keeps every order, even ones referencing a deleted restaurant
SELECT
    o.order_id,
    r.name AS restaurant_name, -- NULL when the restaurant no longer exists
    o.total_amount
FROM orders o
LEFT JOIN restaurants r ON o.restaurant_id = r.restaurant_id;
```

S5.

Justify why a VIEW is preferred over direct table access for the top_restaurants analytics layer. What are two limitations the front-end developer should be aware of when working with this VIEW?

Answer:

A VIEW is preferred here for a few reasons: it hides raw internal ID columns and any underlying join/aggregation complexity from the front-end team, exposing only the clean, business-friendly columns (restaurant name, total orders, average rating) that they actually need. It also centralizes the calculation

logic in one place - if the definition of 'average rating' or 'total orders' ever changes, it's updated once in the VIEW rather than in every query across the codebase. Additionally, it acts as a simple access-control layer, since the front-end team never touches the raw restaurants/orders/ratings tables directly. Two limitations to be aware of: 1. Performance: a VIEW built on JOINS and aggregate functions (like COUNT, AVG) is not materialized by default - MySQL re-runs the underlying query every time the VIEW is queried, so it can be slow on large datasets unless the base tables are well-indexed. 2. Read-only in practice: views involving GROUP BY, aggregate functions, or multi-table joins are generally not updatable in MySQL, so the front-end can only SELECT from top_restaurants - it cannot INSERT, UPDATE, or DELETE through this VIEW.

S6.

Which database mechanism should have prevented the wallet/payment inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

Answer:

The mechanism that should have prevented this is a database transaction - wrapping both the wallet deduction and the payment record insert inside a single atomic unit using START TRANSACTION. Transactions guarantee atomicity: either all statements inside the transaction succeed and are made permanent together, or none of them are applied at all. COMMIT makes every change made since the transaction started permanent and visible to other connections - it should only be called once every step has succeeded. ROLLBACK undoes every change made since the transaction began (or since a specific SAVEPOINT), restoring the database to its earlier consistent state - it should be called the moment any step fails. SAVEPOINT marks a named intermediate point within a longer transaction, letting you roll back part of the work (back to that savepoint) without discarding everything done before it, which is useful in multi-step processes with several independent sub-steps. For this payment module, wrapping the wallet UPDATE and the payment INSERT inside one transaction means that if the payment insert fails for any reason, the whole transaction rolls back - including the wallet deduction - so the customer's balance is never left in an inconsistent, partially-updated state.

```
START TRANSACTION;
```

```
UPDATE customers
SET wallet_balance = wallet_balance - 250
WHERE customer_id = 101;
```

```
SAVEPOINT after_wallet_deduction;
```

```
INSERT INTO payments (customer_id, order_id, amount, payment_date)
VALUES (101, 5001, 250, NOW());
```

```
-- If the INSERT above fails or business logic detects a problem:
-- ROLLBACK TO after_wallet_deduction;    -- or ROLLBACK; to undo everything
```

```
COMMIT;    -- only reached if both statements succeeded
```

Section B – Practical Coding Tasks

Task 1: Design and Populate a Restaurant Database

Create foodapp_db and a restaurants table (restaurant_id, name, city, cuisine_type, rating). Insert 6+ records across 3+ cities. Update one rating, delete one record with WHERE. Select all sorted by rating

descending.

Answer:

```
CREATE DATABASE foodapp_db;
USE foodapp_db;

CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    city          VARCHAR(50),
    cuisine_type  VARCHAR(50),
    rating        DECIMAL(3,1)
);

INSERT INTO restaurants (name, city, cuisine_type, rating) VALUES
('Punjabi Tadka', 'Mumbai', 'Indian', 4.5),
('Mainland China', 'Pune', 'Chinese', 4.2),
('Pizza Express', 'Bengaluru', 'Italian', 4.0),
('Taco Bell', 'Delhi', 'Mexican', 3.8),
('Saffron House', 'Mumbai', 'Indian', 4.7),
('Wok This Way', 'Pune', 'Chinese', 4.1);

-- Update one restaurant's rating
UPDATE restaurants
SET rating = 4.8
WHERE name = 'Saffron House';

-- Delete one restaurant record using a WHERE condition
DELETE FROM restaurants
WHERE name = 'Taco Bell';

-- Select all restaurants sorted by rating descending
SELECT * FROM restaurants
ORDER BY rating DESC;
```

Task 2: Aggregate Revenue Reporting on Orders

Create an orders table and insert 12+ orders across 4+ restaurants. Query total revenue and average order value per restaurant. Query restaurants with revenue over Rs 5,000 using HAVING. Query the 5 most recent orders using ORDER BY and LIMIT.

Answer:

```
CREATE TABLE orders (
    order_id      INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    total_amount  DECIMAL(10,2) NOT NULL,
    order_date    DATE NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

INSERT INTO orders (restaurant_id, customer_name, total_amount, order_date) VALUES
(1, 'Rohan Mehta', 850.00, '2026-09-01'),
(1, 'Sneha Patil', 620.00, '2026-09-05'),
(1, 'Aman Verma', 990.00, '2026-09-12'),
(2, 'Priya Nair', 450.00, '2026-09-02'),
(2, 'Karan Shah', 720.00, '2026-09-08'),
(2, 'Neha Joshi', 530.00, '2026-09-15'),
(3, 'Rahul Singh', 300.00, '2026-09-03'),
(3, 'Divya Rao', 410.00, '2026-09-09'),
(5, 'Anjali Gupta', 1200.00, '2026-09-04'),
(5, 'Vikram Desai', 980.00, '2026-09-10'),
(5, 'Meera Iyer', 1450.00, '2026-09-18'),
(5, 'Sanjay Kapoor', 875.00, '2026-09-20');
```

```
-- Total revenue and average order value per restaurant
SELECT
    restaurant_id,
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS avg_order_value
FROM orders
GROUP BY restaurant_id;

-- Restaurants whose total revenue exceeds Rs 5,000
SELECT
    restaurant_id,
    SUM(total_amount) AS total_revenue
FROM orders
GROUP BY restaurant_id
HAVING SUM(total_amount) > 5000;

-- 5 most recent orders
SELECT * FROM orders
ORDER BY order_date DESC
LIMIT 5;
```

Task 3: Multi-Table JOINS and Menu Summary View

Create restaurants and menu_items (linked by FOREIGN KEY). Insert 5+ restaurants and 12+ menu items, with at least one restaurant having no items. Write an INNER JOIN query and a LEFT JOIN query. Create a VIEW named restaurant_menu_summary.

Answer:

```
CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    city          VARCHAR(50)
);

CREATE TABLE menu_items (
    item_id       INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    item_name      VARCHAR(100) NOT NULL,
    price         DECIMAL(8,2) NOT NULL,
    category       VARCHAR(50),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

INSERT INTO restaurants (name, city) VALUES
('Punjabi Tadka', 'Mumbai'),
('Mainland China', 'Pune'),
('Pizza Express', 'Bengaluru'),
('Saffron House', 'Mumbai'),
('Wok This Way', 'Pune'); -- this restaurant will be given no menu items

INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES
(1, 'Butter Chicken', 320.00, 'Main Course'),
(1, 'Dal Makhani', 220.00, 'Main Course'),
(1, 'Garlic Naan', 60.00, 'Bread'),
(2, 'Hakka Noodles', 250.00, 'Noodles'),
(2, 'Veg Manchurian', 210.00, 'Starter'),
(2, 'Spring Roll', 180.00, 'Starter'),
(3, 'Margherita Pizza', 299.00, 'Pizza'),
(3, 'Pasta Alfredo', 260.00, 'Pasta'),
(3, 'Garlic Bread', 150.00, 'Sides'),
(4, 'Chicken Biryani', 280.00, 'Main Course'),
(4, 'Paneer Tikka', 240.00, 'Starter'),
(4, 'Gulab Jamun', 90.00, 'Dessert');
-- Wok This Way (restaurant_id 5) intentionally has no menu items
-- INNER JOIN: only restaurants that actually have menu items
```

```

SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants r
INNER JOIN menu_items m ON r.restaurant_id = m.restaurant_id;

-- LEFT JOIN: every restaurant appears, NULLs for restaurants with no items
SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants r
LEFT JOIN menu_items m ON r.restaurant_id = m.restaurant_id;

-- VIEW: restaurant menu summary
CREATE VIEW restaurant_menu_summary AS
SELECT
    r.name AS restaurant_name,
    COUNT(m.item_id) AS total_menu_items,
    AVG(m.price) AS avg_item_price
FROM restaurants r
LEFT JOIN menu_items m ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.name;

-- Test the view
SELECT * FROM restaurant_menu_summary;

```

Task 4: PL/SQL Order Placement Procedure

Create a stored procedure `place_order(customer_id, restaurant_id, amount)` that deducts from `wallet_balance` and inserts an order, using a transaction that rolls back if the balance is insufficient. Test with a valid order and one exceeding the balance.

Answer:

```

CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    wallet_balance DECIMAL(10,2) NOT NULL DEFAULT 0
);

INSERT INTO customers (name, wallet_balance) VALUES
('Rohan Mehta', 1000.00),
('Sneha Patil', 200.00);

DELIMITER //

CREATE PROCEDURE place_order (
    IN p_customer_id INT,
    IN p_restaurant_id INT,
    IN p_amount DECIMAL(10,2)
)
BEGIN
    DECLARE current_balance DECIMAL(10,2);

    START TRANSACTION;

    SELECT wallet_balance INTO current_balance
    FROM customers
    WHERE customer_id = p_customer_id
    FOR UPDATE;

    IF current_balance < p_amount THEN
        ROLLBACK;
        SELECT 'Order failed: insufficient wallet balance.' AS result;
    ELSE
        UPDATE customers
        SET wallet_balance = wallet_balance - p_amount
        WHERE customer_id = p_customer_id;

        INSERT INTO orders (restaurant_id, customer_name, total_amount, order_date)

```

```

VALUES (p_restaurant_id,
        (SELECT name FROM customers WHERE customer_id = p_customer_id),
        p_amount, CURDATE());

COMMIT;
SELECT 'Order placed successfully.' AS result;
END IF;
END //

DELIMITER ;

-- Test 1: valid order (balance sufficient)
CALL place_order(1, 1, 350.00);

-- Test 2: order exceeding wallet balance
CALL place_order(2, 1, 500.00);

```

Section C – Mini Capstone Project

Food Delivery Platform - Backend Database

Build three linked tables (restaurants, menu_items, orders) with constraints, populate with sample data, create a restaurant_sales_summary VIEW, a stored procedure add_order with validation, and a TRIGGER after_order_insert that logs to an order_audit table.

Answer:

```

CREATE DATABASE fooddelivery_db;
USE fooddelivery_db;

CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    city          VARCHAR(50) NOT NULL,
    cuisine_type  VARCHAR(50),
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE menu_items (
    item_id       INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    item_name     VARCHAR(100) NOT NULL,
    price        DECIMAL(8,2) NOT NULL,
    category     VARCHAR(50) NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

CREATE TABLE orders (
    order_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    restaurant_id INT NOT NULL,
    item_id       INT NOT NULL,
    quantity      INT NOT NULL,
    total_amount  DECIMAL(10,2) NOT NULL,
    order_date    DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
    FOREIGN KEY (item_id) REFERENCES menu_items(item_id)
);

CREATE TABLE order_audit (
    audit_id      INT AUTO_INCREMENT PRIMARY KEY,
    order_id      INT NOT NULL,
    restaurant_id INT NOT NULL,
    action        VARCHAR(20) NOT NULL,

```

```

        log_time          DATETIME NOT NULL
    );

-- ----- Sample data: 3 restaurants, 4 menu categories, 15 orders -----
INSERT INTO restaurants (name, city, cuisine_type) VALUES
('Punjabi Tadka', 'Mumbai', 'Indian'),
('Mainland China', 'Pune', 'Chinese'),
('Pizza Express', 'Bengaluru', 'Italian');

INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES
(1, 'Butter Chicken', 320.00, 'Main Course'),
(1, 'Garlic Naan', 60.00, 'Bread'),
(2, 'Hakka Noodles', 250.00, 'Noodles'),
(2, 'Spring Roll', 180.00, 'Starter'),
(3, 'Margherita Pizza', 299.00, 'Pizza'),
(3, 'Tiramisu', 180.00, 'Dessert');

INSERT INTO orders (customer_name, restaurant_id, item_id, quantity, total_amount) VALUES
('Rohan Mehta', 1, 1, 1, 320.00), ('Sneha Patil', 1, 2, 2, 120.00),
('Aman Verma', 1, 1, 2, 640.00), ('Priya Nair', 2, 3, 1, 250.00),
('Karan Shah', 2, 4, 3, 540.00), ('Neha Joshi', 2, 3, 2, 500.00),
('Rahul Singh', 3, 5, 1, 299.00), ('Divya Rao', 3, 6, 2, 360.00),
('Anjali Gupta', 1, 1, 1, 320.00), ('Vikram Desai', 2, 4, 1, 180.00),
('Meera Iyer', 3, 5, 2, 598.00), ('Sanjay Kapoor', 1, 2, 4, 240.00),
('Ritu Malhotra', 2, 3, 1, 250.00), ('Arjun Reddy', 3, 6, 1, 180.00),
('Kavya Menon', 1, 1, 1, 320.00);

-- ----- VIEW: restaurant_sales_summary -----
CREATE VIEW restaurant_sales_summary AS
SELECT
    r.name AS restaurant_name,
    COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_revenue
FROM restaurants r
LEFT JOIN orders o ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.name;

SELECT * FROM restaurant_sales_summary;

-- ----- Stored procedure: add_order -----
DELIMITER //

CREATE PROCEDURE add_order (
    IN p_customer_name VARCHAR(100),
    IN p_restaurant_id INT,
    IN p_item_id INT,
    IN p_quantity INT
)
BEGIN
    DECLARE v_price DECIMAL(8,2);
    DECLARE v_restaurant_exists INT;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_restaurant_exists
    FROM restaurants
    WHERE restaurant_id = p_restaurant_id;

    IF v_restaurant_exists = 0 THEN
        ROLLBACK;
        SELECT 'Order failed: restaurant does not exist.' AS result;
    ELSE
        SELECT price INTO v_price
        FROM menu_items
        WHERE item_id = p_item_id AND restaurant_id = p_restaurant_id;

        INSERT INTO orders (customer_name, restaurant_id, item_id, quantity, total_amount)

```

```

VALUES (p_customer_name, p_restaurant_id, p_item_id, p_quantity, v_price * p_quantity);

COMMIT;
SELECT 'Order placed successfully.' AS result;
END IF;
END //

DELIMITER ;

-- Test
CALL add_order('Test Customer', 1, 1, 2);

-- ----- TRIGGER: after_order_insert -----
DELIMITER //

CREATE TRIGGER after_order_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    INSERT INTO order_audit (order_id, restaurant_id, action, log_time)
    VALUES (NEW.order_id, NEW.restaurant_id, 'INSERT', NOW());
END //

DELIMITER ;

-- Verify: every new order should now also produce a matching order_audit row
SELECT * FROM order_audit;
```

Section D – AI-Augmented Learning

1. Exact prompt given to the AI tool

The prompt used to generate the initial stored procedure:

Answer:

"Write a MySQL stored procedure that accepts a restaurant_id and a month number (1-12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month, using SELECT with GROUP BY and aggregate functions. If the restaurant has no orders in that month, return a meaningful message instead of an empty result."

2a. AI's original stored procedure

The first version produced by the AI tool:

Answer:

```

DELIMITER //

CREATE PROCEDURE monthly_restaurant_summary (
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    SELECT
        restaurant_id,
        COUNT(*) AS total_orders,
        SUM(total_amount) AS total_revenue,
        AVG(total_amount) AS avg_order_value
    FROM orders
    WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month
```

```

        GROUP BY restaurant_id;
    END //

DELIMITER ;

```

2b. Corrected version (after testing without AI)

Testing this in MySQL Workbench with real order data revealed two problems: (1) when a restaurant has zero orders in the given month, the query returns an empty result set instead of any message at all, which fails the assessment's explicit requirement; and (2) the procedure filters MONTH(order_date) = p_month across all years, so it would silently combine September 2025 and September 2026 orders together if both existed - there's no year filter. The corrected version adds a p_year parameter and an explicit empty-result check:

Answer:

```

DELIMITER //

CREATE PROCEDURE monthly_restaurant_summary (
    IN p_restaurant_id INT,
    IN p_month INT,
    IN p_year INT
)
BEGIN
    DECLARE v_order_count INT;

    SELECT COUNT(*) INTO v_order_count
    FROM orders
    WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month
        AND YEAR(order_date) = p_year;

    IF v_order_count = 0 THEN
        SELECT CONCAT('No orders found for restaurant ', p_restaurant_id,
            ' in ', p_month, '/', p_year, '.') AS message;
    ELSE
        SELECT
            restaurant_id,
            COUNT(*) AS total_orders,
            SUM(total_amount) AS total_revenue,
            AVG(total_amount) AS avg_order_value
        FROM orders
        WHERE restaurant_id = p_restaurant_id
            AND MONTH(order_date) = p_month
            AND YEAR(order_date) = p_year
        GROUP BY restaurant_id;
    END IF;
END //

DELIMITER ;

-- Test with two different restaurant_id / month combinations
CALL monthly_restaurant_summary(1, 9, 2026);
CALL monthly_restaurant_summary(3, 1, 2026); -- expected: "No orders found..." message

```

3. Note on what was changed and why

Summary of the fix:

Answer:

The AI's original procedure correctly used GROUP BY and aggregate functions to compute the summary, but it silently returned an empty result set for a restaurant with no orders in the given month, which directly

violates the requirement to return 'a meaningful message rather than an empty result.' I added an explicit COUNT(*) check before running the aggregate query, and return a readable message when the count is zero. I also noticed the AI's version had no year parameter, so MONTH(order_date) alone would match a given month across every year in the table - I added a p_year parameter and an additional YEAR(order_date) = p_year condition so the summary is scoped to one specific month and year, not just a repeating calendar month.